INFORME EJECUTIVO

DEADBEAT

Proyecto Videojuego (GM) · Programación Avanzada

Sebastián Arriagada · Joaquín Carrillo · Benjamín Beltrán

Versión documentada: a2f7d4b · Java 11 · LibGDX 1.14.2 · Gradle 7.6.4

Repositorio: https://github.com/Klonoh/Deadbeat

Verificación académica: https://github.com/Klonoh/Deadbeat/actions/runs/43540090728

Verificación portable: https://github.com/Klonoh/Deadbeat/actions/runs/43540090732



Maqueta esquemática del menú. No corresponde a una captura de ejecución.

Resumen ejecutivo

Deadbeat es un videojuego de supervivencia 2D con cinco oleadas, armas musicales, experiencia y mejoras. Se desarrolla desde un código fuente común, compatible con la edición académica (JDK 11) y la distribución portable para Windows (jpackage con JDK 17). El informe detalla los diez criterios GM de la rúbrica con evidencias concretas y diagramas del código actual.

Los dos workflows del commit de referencia aprobaron en GitHub Actions. La verificación académica utiliza Temurin 11 para compatibilidad automatizada; la rúbrica pide Oracle JDK 11, cuya validación en Eclipse debe ejecutarse localmente. La construcción portable se verificó estructuralmente, no mediante una sesión gráfica automatizada.

Matriz de trazabilidad

Criterio	Implementación principal	Evidencia gráfica
GM-1	Menú, HUD, progresión, victoria/derrota	GM1, GM1, GM1
GM-2	Separación de responsabilidades y módulos	GM3
GM-3	LibGDX, clases propias y relaciones	GM3, GM3
GM-4	Weapon + Damageable	GM4
GM-5	Encapsulamiento y composición	GM4
GM-6	AudioSettings Singleton	GM6
GM-7	MovementStrategy	GM7
GM-8	Weapon.update() final	GM8
GM-9	WaveBuilder	GM9
GM-10	Commits y workflows	GitHub

GM-1 · Experiencia del jugador

Deadbeat es un videojuego de supervivencia con vista superior ambientado en una ciudad postapocalíptica de estética punk. El jugador controla a un músico que combate hordas de zombis mediante instrumentos: una guitarra eléctrica que dispara proyectiles y unas baquetas que realizan ataques cercanos. El objetivo es sobrevivir a cinco oleadas, derrotar a todos los enemigos restantes de la quinta y evitar que la vida del protagonista llegue a cero.

El jugador puede desplazarse con WASD o flechas; los ataques de las armas se activan automáticamente. Durante la partida aparecen orbes de experiencia que permiten subir de nivel y elegir mejoras con las teclas 1, 2 y 3 o con el ratón. ESC abre o cierra la pausa; R reinicia la partida; Q y E cambian la pista musical. La tecla H inflige daño al protagonista y se conserva como control de depuración, no como mecánica principal. El menú inicial permite comenzar o salir y las pantallas finales ofrecen reintentar o abandonar.

Las oleadas emplean una duración base de 30 segundos y una cantidad predefinida de enemigos por fase. El final de la quinta oleada no produce una victoria automática si aún quedan zombis vivos. Las figuras GM1 son maquetas explicativas, no capturas tomadas de la ejecución. La rúbrica admite expresamente maquetas o pantallazos.

Evidencia del código

screens/MainMenuScreen.java · líneas 92–179. render(), activate() y controles de menú.

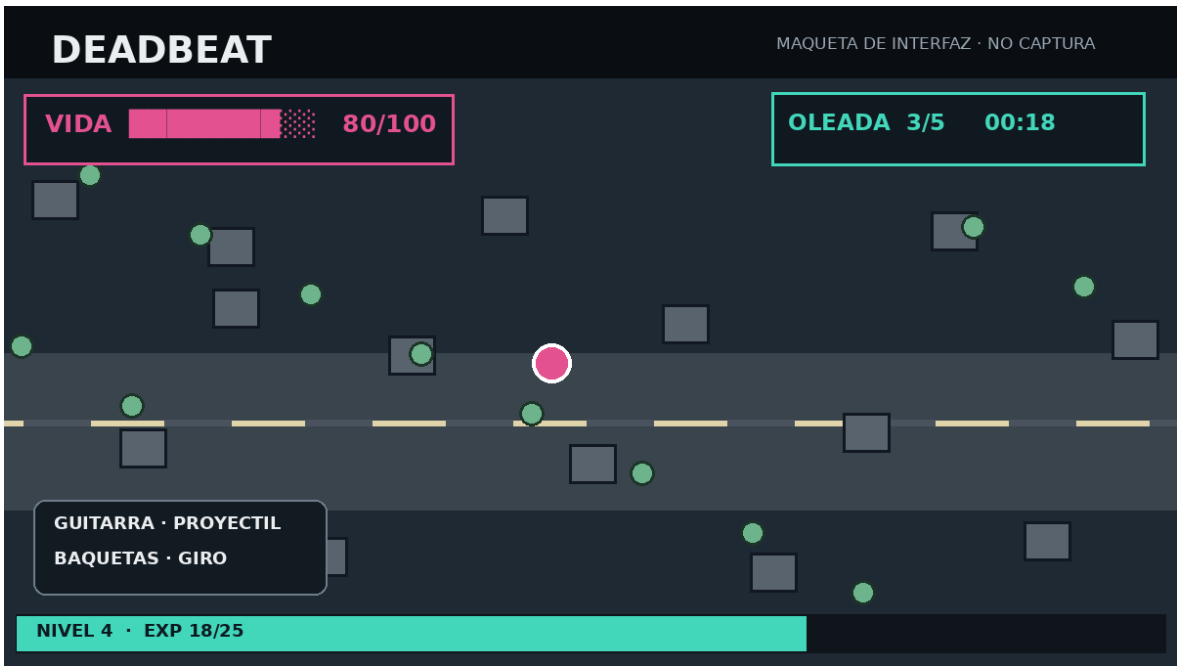
screens/GameScreen.java · líneas 158–332, 535–612. render(), drawExperienceBar(), handleUpgradeInput(), checkRunOutcome(), drawResultScreen().

world/GameWorld.java · líneas 194–251. updateSpawning(), advanceWave(), isChoosingUpgrade().

entities/Player.java · líneas 53–95, 121–209. update(), addExperience(), updateWeapons().



GM1 Maqueta Menu · Maqueta esquemática, no captura real.



GM1 Maqueta Partida · Maqueta esquemática, no captura real.



GM1 Maqueta Victoria · Maqueta esquemática, no captura real.

GM-2 · Análisis desde ingeniería de software

Como referencias conceptuales de organización de juegos Java se identificaron SpaceNav2024 y GameLluviaMenu2024 en el reporte inicial del equipo. Estas referencias se usan para comparar ideas de movimiento, proyectiles y organización de pantallas; el producto desarrollado es Deadbeat, con su propio dominio y mecánicas. No se atribuye código reutilizado a los ejemplos de referencia.

Las principales decisiones de diseño fueron: separación entre core (lógica del juego) y lwjgl3 (lanzador de escritorio); pantallas para menú y partida; mapa urbano con colisiones; entidades independientes; arma abstracta y dos implementaciones; estrategias de movimiento intercambiables; cinco oleadas configuradas con Builder; recompensas de experiencia y selección de mejoras; y controles de audio compartidos mediante Singleton.

GameScreen traduce entradas y presenta el estado; GameWorld coordina la simulación, aparición y eliminación de enemigos, experiencia, colisiones y oleadas; Player y Zombie concentran estado de sus entidades. Esta partición permite modificar la presentación sin reescribir la lógica de combate. Las pruebas en core verifican comportamientos independientes de la ventana gráfica, y los workflows permiten detectar regresiones en Java 11 y en el empaquetado portable.

Comparación de decisiones de diseño

Aspecto	Referencias conceptuales	Deadbeat
Movimiento		
	Ejemplos de movimiento	
		Player, colisiones y cámara
Proyectiles		
	Ejemplos de disparos	

		Guitarra y Projectile
Pantallas		
	Menú y pantalla principal	
		MainMenuScreen y GameScreen
Progresión		
	Sin atribución verificable	
		Experiencia, mejoras y oleadas

Evidencia del código

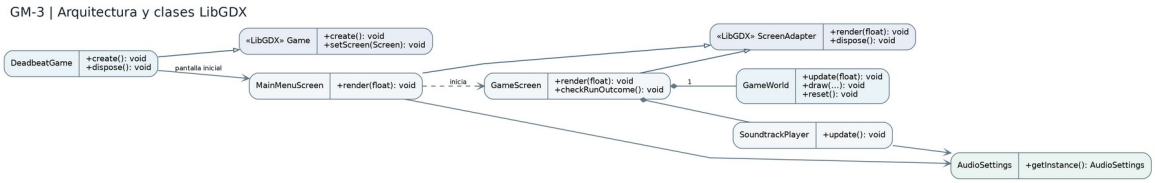
../build.gradle · líneas 1–24. estructura de compilación y plugin de integración Eclipse.

../settings.gradle · líneas 1–2. declaración de los módulos core y lwjgl3.

screens/GameScreen.java · líneas 158–220. orquestación de pantalla y HUD.

world/GameWorld.java · líneas 99–251. simulación, colisiones y oleadas.

movement/MovementStrategy.java · líneas 1–10. extensión del movimiento por interfaz.



GM3 Arquitectura · Diagrama UML del código actual.

GM-3 · Diagrama UML del dominio y clases LibGDX

DeadbeatGame extiende la clase Game de LibGDX y dirige la pantalla inicial a MainMenuScreen. MainMenuScreen y GameScreen extienden ScreenAdapter. GameScreen compone un GameWorld y un SoundtrackPlayer. GameWorld coordina las entidades Player, Zombie, Projectile y ExperienceOrb, las oleadas Wave, el mapa CityMap y la selección de mejoras UpgradeSelection.

En el dominio se distinguen dos contratos principales: Damageable, aplicado por Player y Zombie; y MovementStrategy, que permite a Zombie delegar en ChaseMovement o ZigzagMovement. La abstracción Weapon agrupa el comportamiento de ElectricGuitar y Drumsticks. Las configuraciones Wave se construyen con WaveBuilder. Los diagramas muestran relaciones esenciales del código actual; las fuentes editables se incluyen en uml_fuentes/.

Evidencia del código

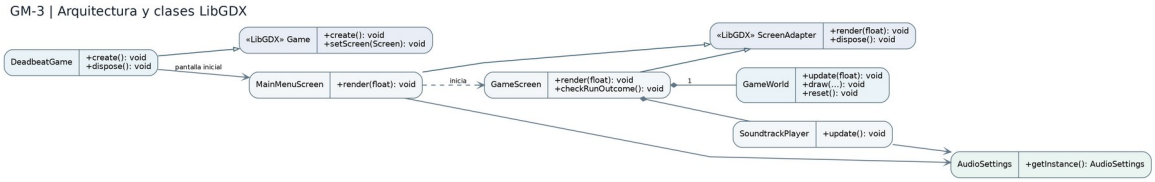
DeadbeatGame.java · líneas 1–20. extends Game, create() y setScreen().

screens/GameScreen.java · líneas 23–155. extends ScreenAdapter y composición de GameWorld.

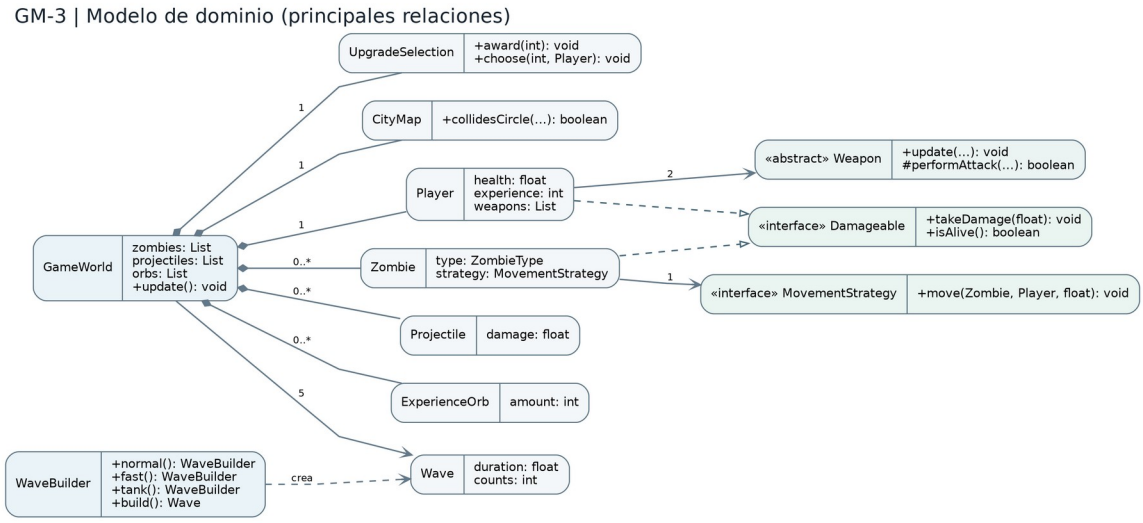
entities/Player.java · líneas 23–51. implements Damageable; List<Weapon>.

entities/Zombie.java · líneas 11–56. implements Damageable; MovementStrategy.

world/GameWorld.java · líneas 28–96. composición de entidades y waves.



GM3 Arquitectura · Diagrama UML del código actual.



GM3 Dominio · Diagrama UML del código actual.

GM-4 · Clase abstracta e interfaz con dos implementaciones

La clase abstracta Weapon es la generalización de dos clases utilizadas durante una partida: ElectricGuitar y Drumsticks. Player crea ambas armas al reiniciar, conserva referencias del tipo base Weapon y ejecuta su actualización polimórfica. Weapon define un método abstracto performAttack() y un método update() compartido.

La interfaz Damageable declara takeDamage(float) e isAlive(). Player y Zombie la implementan, y GameWorld utiliza ese contrato en el método damage(Damageable, float). Ambos requisitos se observan en el juego y no se limitan a clases declaradas sin uso.

Evidencia del código

weapons/Weapon.java · líneas 7–24. clase abstracta y performAttack() abstracto.

weapons/ElectricGuitar.java · líneas 6–20. primera subclase concreta.

weapons/Drumsticks.java · líneas 6–14. segunda subclase concreta.

entities/Damageable.java · líneas 4–10. interfaz Damageable.

entities/Player.java · líneas 23–49, 239–257. implements Damageable, List<Weapon>, reset().

entities/Zombie.java · líneas 11–20. implements Damageable.

world/GameWorld.java · líneas 293–297. damage(Damageable, float).



GM4 Abstraccion Interfaz · Diagrama UML del código actual.

GM-5 · Encapsulamiento y principios de OO

El estado mutable de las entidades se almacena en atributos privados. Player encapsula coordenadas, salud, nivel, experiencia y armas. Los métodos getWeapons() y los getters de GameWorld proporcionan vistas no modificables de sus listas, de modo que las demás clases no alteran directamente la composición interna del mundo.

La solución utiliza encapsulamiento, herencia y polimorfismo (Weapon y sus dos subclases), contratos mediante interfaces (Damageable y MovementStrategy) y composición (GameWorld con sus entidades). Las responsabilidades de presentación y simulación permanecen diferenciadas: GameScreen gestiona controles y HUD, mientras GameWorld mantiene las reglas de combate y oleadas.

Evidencia del código

entities/Player.java · líneas 27–51, 84–95. atributos privados, getWeapons() y control de estado.

entities/Zombie.java · líneas 11–50, 126–130. atributos privados y estrategia encapsulada.

world/GameWorld.java · líneas 52–55, 216–218, 341–343. listas privadas y getters de sólo lectura.

weapons/Weapon.java · líneas 7–24. abstracción y actualización compartida.



GM4 Abstraccion Interfaz · Diagrama UML del código actual.

GM-6 · Singleton — AudioSettings

Problema: el menú principal, la partida y el reproductor de música necesitan aplicar una sola configuración de volumen y silencio. Si cada pantalla almacenara su propio volumen, se producirían cambios inconsistentes al entrar y salir de una partida.

Contexto: Deadbeat dispone de un menú con ajustes de volumen y una pantalla de partida que reproduce pistas. La configuración debe mantenerse cuando se crea una pantalla o se vuelve al menú.

Solución: AudioSettings es una clase final con constructor privado, una instancia estática final (INSTANCE) y un único punto de acceso público getInstance(). Conserva volume y muted, valida el intervalo de volumen y ofrece getEffectiveVolume(). MainMenuScreen, GameScreen y SoundtrackPlayer acceden a la misma instancia. Este es el Singleton implementado en el código; GameSession, mencionada como propuesta en el informe anterior, no forma parte de esta versión.

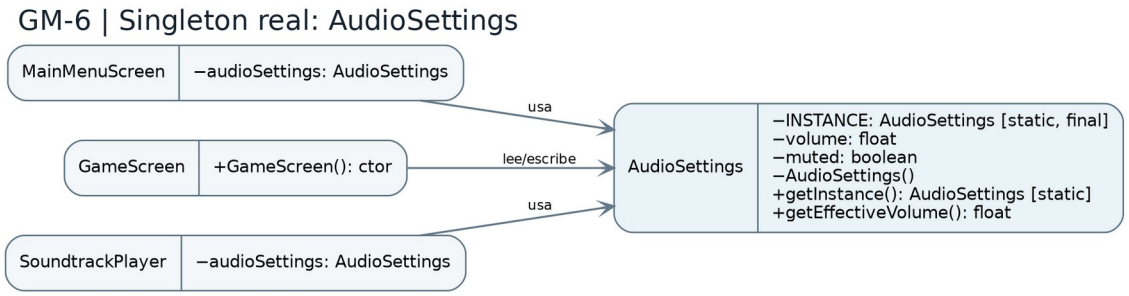
Evidencia del código

audio/AudioSettings.java · líneas 4–39. INSTANCE, constructor privado, getInstance(), volumen y mute.

screens/MainMenuScreen.java · líneas 55, 233–279. instancia AudioSettings y controles de sonido.

screens/GameScreen.java · líneas 112–154. lectura de configuración al iniciar partida.

audio/SoundtrackPlayer.java · líneas 43, 198–219. instancia compartida y aplicación de volumen.



GM6 Singleton · Diagrama UML del código actual.

GM-7 · Strategy — desplazamiento de zombis

Problema: distintos tipos de zombis necesitan algoritmos de movimiento diferentes. Concentrarlos en Zombie con múltiples condicionales dificultaría evolucionar el comportamiento.

Contexto: los enemigos normales y tanque persiguen directamente al protagonista; el enemigo rápido utiliza además zigzag y puede alternar la estrategia durante la simulación.

Solución: MovementStrategy define move(Zombie, Player, float); ChaseMovement y ZigzagMovement concretan las variantes. Zombie almacena una referencia MovementStrategy y delega las decisiones de desplazamiento. El método setMovementStrategy() permite sustituir la estrategia sin crear otra entidad.

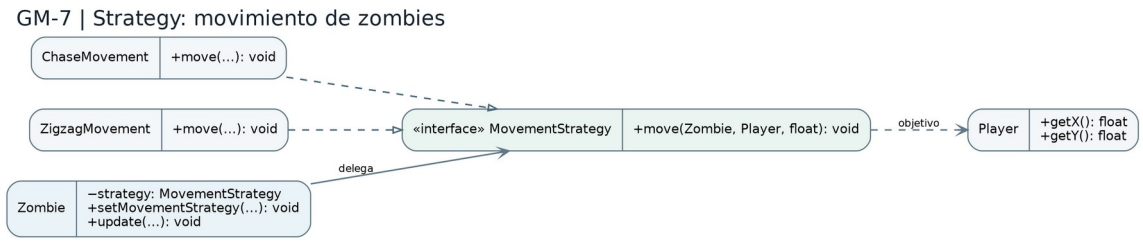
Evidencia del código

movement/MovementStrategy.java · líneas 4–8. interfaz move().

movement/ChaseMovement.java · líneas 5–9. estrategia de persecución.

movement/ZigzagMovement.java · líneas 5–11. estrategia zigzag.

entities/Zombie.java · líneas 11–74, 126–130. selección, alternancia y delegación.



GM7 Strategy · Diagrama UML del código actual.

GM-8 · Template Method — ciclo de armas

Problema: ambos instrumentos deben respetar un intervalo de ataque y descartar actualizaciones inválidas, pero generan ataques diferentes. Duplicar esa secuencia incrementaría la probabilidad de inconsistencias.

Contexto: Player mantiene una colección de Weapon y llama updateWeapons() en cada avance de simulación.

Solución: Weapon implementa update() como método final. La secuencia común valida el delta y la vida, descuenta el tiempo restante e invoca el método abstracto performAttack(). Si el ataque fue efectivo, reinicia el intervalo. ElectricGuitar implementa la creación de un proyectil y Drumsticks ejecuta un ataque cercano. Las subclases redefinen sólo el paso variable.

Evidencia del código

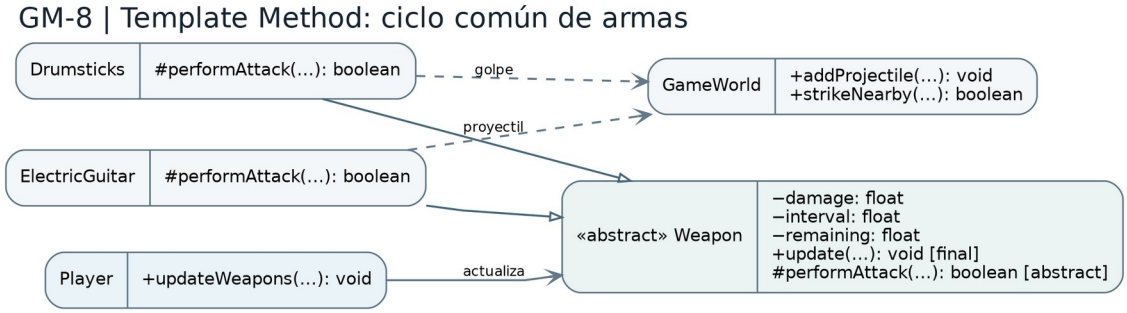
weapons/Weapon.java · líneas 15–24. final update() y abstract performAttack().

weapons/ElectricGuitar.java · líneas 6–20. ataque de proyectil.

weapons/Drumsticks.java · líneas 6–14. ataque cercano.

entities/Player.java · líneas 84–95. recorrido polimórfico updateWeapons().

world/GameWorld.java · líneas 322–340. addProjectile() y strikeNearby().



GM8 Template Method · Diagrama UML del código actual.

GM-9 · Builder — configuración de oleadas

Problema: cada oleada necesita combinar tiempo de duración y tres cantidades de zombis. Un constructor con muchos parámetros posicionales dificulta la lectura y hace probable un intercambio de valores.

Contexto: el juego configura cinco oleadas crecientes. Al iniciar GameWorld, es necesario disponer de objetos Wave con configuraciones validadas.

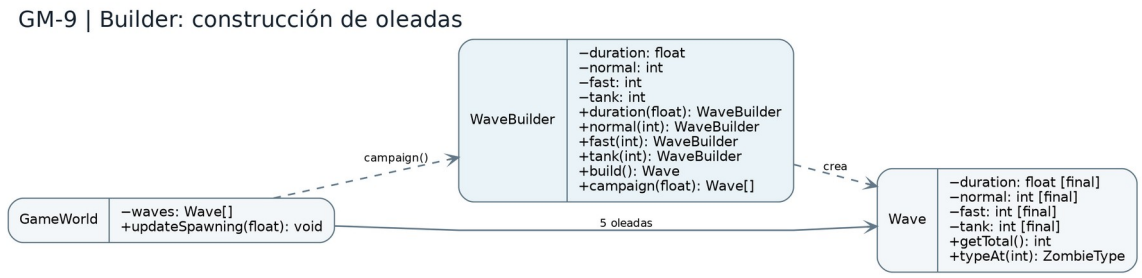
Solución: WaveBuilder admite llamadas encadenadas duration(), normal(), fast() y tank(), y build() valida la configuración antes de crear un Wave. campaign() genera las cinco oleadas, con totales de 24, 34, 48, 64 y 82 zombis (252 en conjunto). Wave es el objeto final que GameWorld utiliza para dosificar las apariciones según el tiempo y la capacidad simultánea.

Evidencia del código

progression/WaveBuilder.java · líneas 3–47. métodos encadenados, build(), campaign().

progression/Wave.java · líneas 6–39. objeto final con getTotal() y typeAt().

world/GameWorld.java · líneas 68–74, 220–251. creación de waves y actualización de spawns.



GM9 Builder · Diagrama UML del código actual.

GM-10 · Control de versiones y trazabilidad

Deadbeat se desarrolla en el repositorio GitHub Klonoh/Deadbeat. El historial contiene más de los seis commits exigidos por la rúbrica y registra iteraciones de mecánicas, recursos, compatibilidad Java 11, integración con Eclipse, empaquetado portable y reparación del sistema de mejoras.

Para el corte documentado se utiliza el commit a2f7d4b. En ese commit las dos ejecuciones de GitHub Actions finalizaron con éxito: Académico JDK 11 (Gradle, JUnit y bytecode 11) y Windows portable JDK 17 (JUnit, empaquetado y verificación estructural). El flujo académico usa Temurin 11 para verificar compatibilidad; la comprobación literal con Oracle JDK 11 en Eclipse debe realizarse en el equipo de entrega. La verificación estructural del ZIP tampoco reemplaza una prueba de jugabilidad en Windows.

Evidencia del código

../../.github/workflows/deadbeat-academico-jdk11.yml · líneas archivo completo. verificación automática de compatibilidad y pruebas.

../../.github/workflows/deadbeat-portable-windows.yml · líneas archivo completo. verificación automática y construcción de ejecutable.

../../.git/ · líneas GitHub /commits/main. historial de commits y colaboradores.

Anexos existentes y comprobaciones finales

Se conservan los PDF existentes en la raíz del repositorio: Anexo_A_GM3_UML_Completo.pdf y Anexo_B_GM6_Singleton.pdf.pdf. Sus contenidos visuales deben contrastarse con la versión actual antes de fusionarlos en la entrega definitiva.

El reporte anterior REPORTE EJECUTIVO DE CUMPLIMIENTO.md también se conserva para trazabilidad histórica. No debe entregarse como versión final sin corregir los marcadores y la referencia obsoleta a GameSession.

Las fuentes de diagramas son editables (Graphviz DOT y PlantUML en uml_fuentes/). Las imágenes renderizadas están en figuras/ junto con las maquetas de GM-1.

Validaciones pendientes: Oracle JDK 11 real en Eclipse; prueba manual del EXE en otro Windows; capturas reales si existen; revisión de derechos de distribución de la música.

Estado de verificaciones

Comprobación	Estado	Observación
Gradle / JUnit / bytecode Java 11	Aprobado en CI	Workflow académico · Temurin 11
Gradle / JUnit / paquete portable Windows	Aprobado en CI	Workflow Windows · Temurin 17
Ejecución local desde Eclipse con Oracle JDK 11	Pendiente de evidencia	Comprobar java -version y pantalla inicial
Ejecución del ZIP portable en Windows sin Java instalado	Pendiente de evidencia	Probar menú, oleadas, mejoras y pantalla final
Compatibilidad visual de anexos históricos y UML revisado	Pendiente de revisión	Conservar sólo diagramas que describan el código
Derechos de la música para distribución pública	Pendiente de revisar	Validar licencias o sustituir pistas

Las líneas de código y las figuras se corresponden con el commit a2f7d4b. Si el código cambia, actualizar esta matriz y volver a verificar las relaciones UML.